City Scan Technical Reference

World Bank - City Resilience Program

2026-05-05

Table of contents

Welcome

The City Scan is a diagnostic tool designed to equip cities with the early insights needed to address risk by:

- Providing a rapid, data-driven snapshot of a city's demographic, economic, climate, and risk landscape.
- Leveraging remote sensing and geospatial analysis of any city in the world, using global datasets.
- Revealing the interaction between the urban built and natural environments.

In this technical reference

This reference book is a structured technical documentation to use and operate City Scan as a diagnostic tool. Here, you will find documentations on City Scan overview, getting started, running a City Scan, tasks and data layers, repository structure, case studies & galleries, as well as contributing guides.

About City Resilience Program (CRP)

The City Resilience Program is housed within the Global Facility for Disaster Reduction and Recovery (GFDRR) at the World Bank. CRP is a global initiative supported by the Swiss State Secretariat for Economic Affairs, the Austrian Federal Ministry of Finance, the Italian Ministry of Foreign Affairs, the Italian Agency for Development Cooperation, and the Gates Foundation.

To learn more about CRP and GFDRR visit https://www.gfdrr.org/en/crp.

Part I

City Scan Overview

1 What is City Scan?

Figure 1.1: City Scan Concept

A City Scan is a rapid geospatial assessment of a city's demographic, socioeconomic, climate, and risk conditions. It is a collection of maps and charts made from global and publicly available datasets that provide quick high-level insights into resilience-related topics for a city. The output is a package of maps, data visualizations and insights that integrate features of both the built and natural environments. These scans are especialy useful for grounding early-stage conversations in geospatial data.

1.1 What can the City Scan tell me?

This City Scan is intended to support operational teams in building dialogue around a city's most pressing resilience challenges. It is designed as a conversation starter rather than a specific decision making tool. Thinking spatially about how urbanization affects the resilience of various urban forces, networks, and people helps equip officials to develop risk informed investment proposals, identify opportunities and barriers to unlocking private capital, and prioritize and coordinate future investments.

1.2 Guiding questions to keep in mind

- How might investment priorities differ in different parts of the city based on this data?
- What spatial relationships are surprising?
- What questions does this spatial pattern spur?
- Where else have you worked that is reminiscent of these findings?
- What might be applicable from there? What might not be?

1.3 Why invest in urban resilience

Investing in urban resilience boosts long-term economic, social, and physical sustainability by safeguarding the gains from current developments for future generations. Greater urban resilience builds the capacity of local governments to help households, communities, and enterprises manage and avoid shocks and stresses, for example by damage-proofing infrastructure before a disaster or maintaining critical services afterward. This approach represents a significant shift from prior development trends, which concentrated investments on post-disaster reconstruction and recovery.

> **Note**
>
> For a hands-on introduction, see Running Your First City Scan.

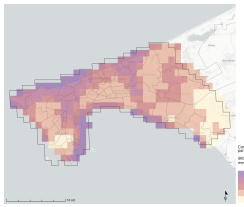
2 City Scan Examples

City Scan output is a collection of maps and charts made from global and publicly available datasets that provide quick high-level insights into resilience-related topics for a city. The output is a package of maps, data visualizations and insights that integrate features of both the built and natural environments. It is available as standalone images, compiled PDF, and interactive Quarto HTML.

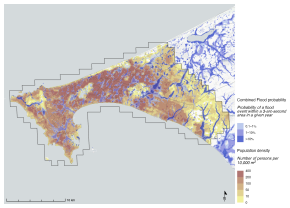
2.1 Example 1: Standalone Images - Dakar, Senegal

This format is useful if you'd like to combine City Scan outputs with your own analysis. See examples from our analysis for Dakar, Senegal

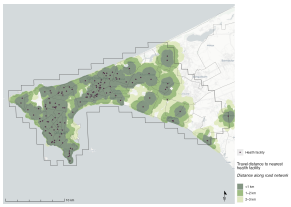
(a) Area of interest



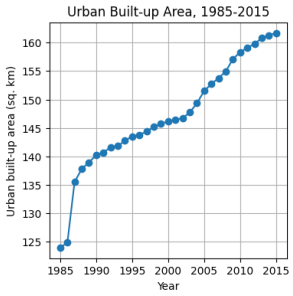
(b) local air quality



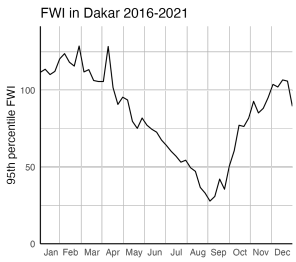
(c) Population exposed to flooding



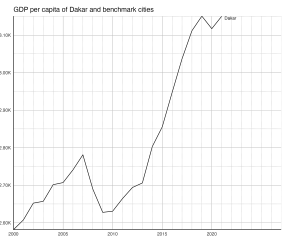
(d) Proximity to health-care infrastructure



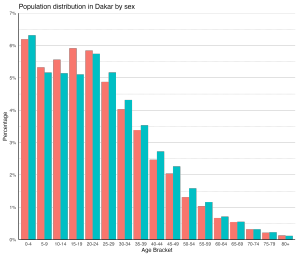
(e) Built-up area over time



(f) Fire weather index



(g) GDP per capita



(h) Age pyramid

Figure 2.1: Examples of maps & charts generated by City Scan

2.2 Example 2: Comprehensive Analytical PDF Report - Bitung, Indonesia

The generated maps and charts are compiled according to relevant themes, along with data interpretation and considerations. See examples below from our analysis in Bitung, Indonesia.

> **Note**
>
> The final PDF output is compiled and designed via Adobe InDesign and not yet automatically generated by the code. Automated PDF generation will be part of future updates.

2.3 Example 3: Interactive HTML website - City, Country

Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt.

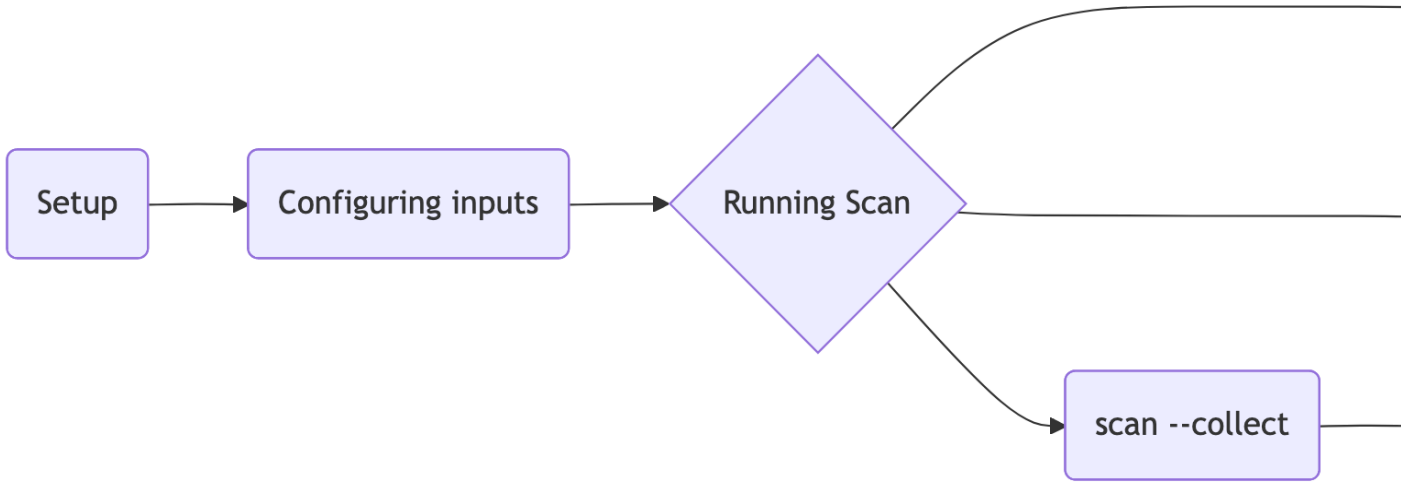
> **Tip**
>
> Full case studies for Rabat and the Philippines are in Part 6. Case Studies & Galleries.

Part II

Standard Workflow

3 General Workflow

This chapter provides a three-part walkthrough of the City Scan repo for new users. The diagram below illustrates the general workflow of performing a standard City Scan.



3.1 Part 1. Basic Setup

> **Note**
>
> See Part 4. Getting Started/ Setup Instructions for full details of prerequisites and installations

City Scan uses multiple programming languages and software to run, including Git, Python, R, Quarto, and Google Cloud. The steps required for setups are below:

- Clone the repo `https://github.com/cityresilience/city-scan`
- Create conda env + activate cityscan
- Installing City Scan as Python Package via `pip install -e .` for the scan CLI

- GCS and GEE authentication via `gcloud auth application-default login`
- Install Quarto
- Run `scan --check` to verify everything passes

3.2 Part 2. Configuring Inputs

Inputs folder is where you can specify your area of interests (AOI), types of analysis to conduct, and certain global analysis parameters.

- Drop AOI shapefile into `inputs/AOI/`
- Edit `inputs/city_inputs.yml`
- Edit `inputs/menu.yml`

3.3 Part 3. Running a Scan

When running a scan, we are essentially going through several phases. The general flow is collect → analyze → multianalysis → render.

1. **Collect** stage is retrieveing specified datasets as indicated in inputs configuration from various publicly available sources for the specified AOI. As an output, you will retrieve TIFFs, GeoJSONs, or Geopackages in your output folder.

   To conduct **collect** stage, run in your terminal

   ```
   scan --collect
   ```

2. **Analyze** stage is conducting various analysis on a single task, such as seeing the statistical distribution, applying threshold, extracting median/mean values, etc. As an output, you will retrieve various CSVs in your output folder.

   To conduct **analyze** stage, run in your terminal

   ```
   scan --analyze
   ```

3. Alternatively, you can **batch data collection and data analysis together by looping over tasks** by running:

   ```
   scan --all
   ```

   Or, you can leverage your computer memory and **parallelizing data collection and data analysis together per tasks** by running:

   ```
   scan --all --parallel
   ```

> ⚠ Warning
>
> [**Internal Note**] Perhaps in the future, `scan --all` and `scan --all --parallel` should be reserved to orchestrate the whole process, including multianalysis, render, and scan-calculations

4. **Multianalysis** stage is performing cross-task analysis after single tasks analysis have been executed. For example, this stage executes analysis of flood-exposed population, which requires both flood analysis and population analysis to have been conducted. To do this, run in your terminal:

   ```
   scan --multianalysis
   ```

5. **Render** stage is generating various visualizations as results of previous stages. This includes rendering maps and charts as PNGs or HTMLs.

   ```
   # Static maps - applies styling from source/layers.yml, writes PNGs to 03-render-output/
   scan --render maps
   ```

   ```
   # Charts for a specific task - writes HTML to tasks/{name}/charts/index.html
   scan --render charts elevation
   ```

   To generate a compiled interactive quarto document and HTML of all tasks, run:

   ```
   scan --render scan-calculations
   ```

4 Setup Instructions

City Scan is multilingual: Python handles data collection and analysis, R generates maps and charts, and Quarto compiles reports and web visualizations. The `scan` CLI ties it all together.

Work through the sections below in order. The checklist from General Workflow:

- ☐ Install prerequisites (VS Code, Git, Python, R, Quarto, gcloud)
- ☐ Clone the repository
- ☐ Create and activate the `cityscan` conda environment
- ☐ Install City Scan as a Python package via `pip install -e .`
- ☐ Authenticate with GCS and GEE
- ☐ Run `scan --check` to verify everything passes

4.1 Step 1 — VS Code (Recommended)

VS Code is a free code editor with an integrated terminal and Git UI. It handles Python, R, and Quarto in the same window, which makes the City Scan workflow much smoother.

Install from code.visualstudio.com.

Recommended extensions — install from the Extensions panel (`Ctrl+Shift+X` / `⌘ X`):

Extension	Purpose
Python	Python language support and environment selection
R	R language support
Quarto	Quarto rendering and preview
Rainbow CSV	Color-coded columns for CSVs under `02-process-output/tabular/`
Color Highlight	Inline color swatches for hex codes in `source/layers.yml`
Geo Data Viewer	Preview `.geojson` and `.shp` files on a map

Extension	Purpose
GeoTIFF Viewer	Preview `.tif` rasters directly in the editor

The following steps require you to use the terminal. In VS Code topbar, go to `Terminal > New Terminal`

4.2 Step 2 — Git

Git is a version control system that tracks changes and enables collaboration through GitHub. It is required to clone the repository and stay up to date with changes.

Git is pre-installed on most macOS and Linux systems. Verify with:

```
git --version
```

If not installed, follow the GitHub Git installation guide. Windows users can use Git for Windows.

4.3 Step 3 — Clone the Repository

```
git clone https://github.com/cityresilience/city-scan
cd city-scan

# switch to unified branch
git switch unified
```

All subsequent steps assume you are inside the `city-scan/` directory.

4.4 Step 4 — Python Environment

City Scan requires Python 3.11. Choose one of the two options below.

4.4.1 Option A: Conda (Recommended — Mac, Windows, Linux)

4.4.1.1 4A-1. Install Anaconda

Download and install from anaconda.com/download. Accept the default settings and restart your terminal after installation.

Verify:

```
conda --version
```

> **`conda: command not found` after install?**
>
> Conda was installed but not initialized for your shell. Run the appropriate command once:
>
> ```
> conda init zsh        # macOS / Linux with zsh (default on modern macOS)
> conda init bash       # Linux / older macOS / WSL
> conda init powershell # Windows PowerShell
> ```
>
> Close and reopen the terminal. You should see `(base)` in your prompt.

4.4.1.2 4A-2. Create and Activate the Environment

```
conda create -n cityscan python=3.11.8 -y
conda activate cityscan
```

4.4.1.3 4A-3. Install City Scan & requirements

```
python -m pip install --upgrade pip
pip install -e .
pip install -r requirements.txt
```

4.4.1.4 4A-4. Register Jupyter Kernel (Optional)

Only needed if you plan to use Jupyter notebooks:

```
python -m ipykernel install --user --name cityscan --display-name "Cityscan"
```

4.4.2 Option B: Bootstrap Script (Mac and Linux only)

Use this option if you prefer automated system setup or do not have Python installed.

4.4.2.1 4B-1. Run the Bootstrap Script

This installs Python 3.11.8 via pyenv and prepares your system:

```
bash bootstrap.sh
```

> **Note**
>
> Review `bootstrap.sh` before running. It installs system dependencies, pyenv, and Python 3.11.8, and updates your shell configuration. If pyenv is being installed for the first time, restart your terminal and run the script again.

4.4.2.2 4B-2. Run the Setup Script

Creates the virtual environment, installs dependencies, and registers the Jupyter kernel:

```
bash orchestrator.sh
```

When successful, you will see:

- Python 3.11.8 ready via pyenv
- A new virtual environment created
- A Jupyter kernel registered: "Cityscan (Python 3.11)"

To activate the environment manually later:

```
source venv/bin/activate
```

4.5 Step 5 — R

R is used for maps, charts, and the scan-calculations report, and also for several data collection tasks (earthquake, Oxford, cyclones, flood events, GDP gridded, coastal erosion, sea level rise).

Download the **latest stable R release** (4.5.x or newer) from cran.r-project.org. The repo depends on a current `ggplot2`, `terra`, `sf`, and `tidyterra` stack that will not install cleanly on older R versions.

Verify:

```
R --version
```

R in VS Code: Install the R extension and, optionally, radian for a richer R console experience.

> **Note**
>
> RStudio works fine for R-specific work and can be downloaded from posit.co. For the full City Scan workflow (Python + R + terminal), VS Code is recommended.

4.6 Step 6 — Quarto

Quarto is the publishing system used to compile City Scan reports and the web visualization. It supports Python, R, JavaScript, and Julia in the same document.

Install from quarto.org/docs/get-started.

Verify:

```
quarto check
```

4.7 Step 7 — Google Cloud (gcloud)

City Scan uses Google Cloud for two services:

- **Google Earth Engine (GEE)** — satellite imagery and geospatial analysis
- **Google Cloud Storage (GCS)** — private data buckets (`city-scan-global-data`, `city-scan-gcs-data`)

If you do not yet have access to these resources, contact the project lead to be added to the Google Cloud project.

4.7.1 7-1. Install gcloud CLI

Download and install from cloud.google.com/sdk/docs/install. After installation, initialize the SDK:

```
gcloud init
```

Verify:

```
gcloud --version
```

4.7.2 7-2. Authenticate

```
gcloud auth application-default login
```

This opens a browser window to complete the OAuth flow. Credentials are stored locally and picked up automatically by City Scan:

- **macOS / Linux**: `~/.config/gcloud/application_default_credentials.json`
- **Windows**: `%APPDATA%\gcloud\application_default_credentials.json`

4.7.3 7-3. Set GEE Project (Optional)

The default GEE project is `city-scan-gee-test`. To override:

```
export GEE_PROJECT=your-project-id
```

4.7.4 7-4. Verify Cloud Access

```
# Check GEE
python -c "import ee; ee.Initialize(); print('GEE OK')"

# Check GCS
gcloud storage ls gs://city-scan-global-public/ --limit=1
```

4.7.5 How Authentication Works in City Scan

Authentication is managed in `core/config/`:

File	Role
`auth.py`	Initializes GEE and GCS credentials
`scan.py`	Loads AOI, city config, and creates output folders
`paths.py`	Auto-detects input/output directories
`gdal_auth.py`	Configures GDAL for access to private GCS buckets via `/vsigs/`

Tasks that require GEE (forest, landcover, LST, NDVI, nightlight) or private GCS (WSF, fathom, landcover burn, basic info) are authenticated automatically. If authentication fails, those tasks are skipped with a warning.

> ⚠ Warning
>
> Never commit credentials or JSON key files to the repository. Treat them like passwords.

4.7.6 Granting Access to a New Team Member

1. Go to IAM & Admin in the Google Cloud Console
2. Click **Grant access**
3. Enter the new user's Gmail address under **New principals**
4. Assign the role **Editor**
5. Click **Save**

The new user then runs the following to authenticate locally:

```
gcloud auth login
gcloud auth application-default login
```

4.7.7 Troubleshooting

Symptom	Fix
GEE auth fails	Re-run `gcloud auth application-default login`
GCS 403 / 404 error	Confirm the account has the **Storage Object Viewer** role on the relevant bucket
"GCS credentials not found"	Check that `~/.config/gcloud/application_default_credentials.jso` exists

4.8 Step 8 — Verify Setup

Run the built-in check from the repo root:

```
scan --check
```

This verifies that Python dependencies, R packages, Quarto, and cloud credentials are all in place. Fix any reported issues before running a scan.

5 Inputs Configuration

The `inputs/` folder controls everything about a scan: which city, which AOI, which tasks run, and the parameters for each task.

Three YAML files drive the pipeline:

File	Purpose
`city_inputs.yml`	Single-city config: city name, AOI, year ranges, flood/accessibility/benchmark params
`multi_inputs.yml`	Multi-city batch config: list of cities (or a multipolygon file) + shared settings
`menu.yml`	Per-task on/off toggles

Templates live in `templates/`. Copy them to `inputs/` and edit before running.

5.1 `city_inputs.yml`

5.1.1 Identification

Key	Type	Description
`city_name`	string	Display name. Spaces and accents OK (`Lobito Corridor`, `Lüderitz`). Used for outputs, labels, and slugified scan_id.

Key	Type	Description
`AOI_shp_name`	string (optional)	Stem of the AOI shapefile in `inputs/AOI/` (no extension). If omitted, AOI is auto-detected from UCDB / OSM / 5km buffer around the city centroid.
`prev_run_date`	string `YYYY-MM` (optional)	Reuse a previous scan folder. Default: `null` (creates fresh `mnt/{YYYY-MM}-{country}-{city}/`).

5.1.2 Time ranges

Key	Type	Description
`first_year`	int	Start year for LST and time-series tasks. Sentinel-2 / Landsat data available from 2013-03-18.
`last_year`	int	End year for the same.
`fwi_first_year`	int	Start year for Fire Weather Index.
`fwi_last_year`	int	End year for FWI.
`demographics_year`	int (optional)	WorldPop R2025A year (2015–2030). Default: current year.

5.1.3 Accessibility (`osm_query` + `isochrone`)

OSM POI categories to fetch and walking-distance buffers for each.

```yaml
osm_query:
  schools:
    amenity: [school, kindergarten, university, college]
  health:
    amenity: [clinic, hospital, health]
  police:
    amenity: [police]
  fire:
```

```
    amenity: [fire_station]

isochrone:  # meters
  schools: [800, 1600, 2400]
  health: [1000, 2000, 3000]

accessibility_buffer: 5   # % of AOI bounds to buffer for capturing amenities just outside th
```

5.1.4 Flood (`flood`)

Drives the Fathom flood analysis.

```
flood:
  threshold: 15           # cm - minimum depth to count as flooded
  year: [2020, 2050, 2100]
  ssp: [2, 5]             # available: 1, 2, 3, 5
  return_period: [10, 100, 1000]   # available: 5, 10, 20, 50, 100, 200, 500, 1000
```

5.1.5 Benchmarks (`benchmark_*`, `bm_cities_*`, `nearby_countries`)

Controls which cities to benchmark against for population/Oxford comparisons.

Key	Values	Description
`benchmark_mode`	`sibling` / `oxford` / `auto` (default)	`sibling` = only sibling scans; `oxford` = only Oxford auto-detect; `auto` = both.
`bm_cities_manual`	list of city names	Explicit benchmark cities. Resolved as sibling → Oxford → backup, in that order.
`benchmark_backup`	`citypopulation` / `worldpop_ucdb` / `null`	Fallback for manual cities not found as sibling or Oxford.
`nearby_countries`	pipe-separated country names	Filter for Oxford auto-detect (only relevant when mode includes Oxford).

5.2 `multi_inputs.yml`

Same fields as `city_inputs.yml` plus the cities/multipolygon list. Two modes — use one (or combine, see below).

5.2.1 Mode A: per-city AOI folders

Each city has its own `inputs/AOI/{city_name}/` folder with a `.shp`. Wards auto-detected from `inputs/AOI/{city_name}_wards/` if present.

```yaml
cities:
  - city_name: Windhoek
  - city_name: Swakopmund
  - city_name: Walvis Bay
  - city_name: Lüderitz
  - city_name: Aus
```

Per-city overrides supported:

```yaml
cities:
  - city_name: Windhoek
    first_year: 2010      # overrides the shared first_year for this city only
  - city_name: Aus
```

5.2.2 Mode B: multipolygon file

A single vector file (GPKG, SHP, GeoJSON) where each row is a city. AOIs are auto-extracted to `inputs/AOI/{city_slug}/`.

```yaml
multipolygon:
  file: inputs/AOI/lobito_corridor_cities.gpkg
  name_column: GC_UCN_MAI_2025
```

5.2.3 Mode B + filter

Combine `multipolygon:` with a `cities:` list to extract all AOIs but only run a subset.

```yaml
multipolygon:
  file: inputs/AOI/lobito_corridor_cities.gpkg
  name_column: GC_UCN_MAI_2025

cities:
  - city_name: Sakania
  - city_name: Lukuni
```

If a name in `cities:` isn't found in the multipolygon file, the run errors out.

5.2.4 Shared settings

All other `city_inputs.yml` keys (flood, accessibility, FWI, year ranges, benchmark_*) work identically here and apply to every city in the list.

5.3 `menu.yml`

Per-task on/off. Set `True` to run, `False` to skip. Tasks not listed default to `False`.

```yaml
accessibility: True
air_quality: True
basic_info: True
buildings: True
burned_area: True
coastal_erosion: True
cyclones: True
demographics: True
earthquake: True
elevation: True
flood_coastal: True
flood_fluvial: True
flood_pluvial: True
flood_comb: True
flood_events: True
forest: True
fwi: True
ghs_builtup: True
```

```yaml
ghs_population: True
green: True
gdp_gridded: True
gdp_sectoral: True
groundwater: True
landcover: True
landcover_burn: True
landslide: True
liquefaction: True
lst_summer: True
lst_winter: True
ndmi: True
nightlight: True
oxford: True
population: True
rwi: True
sea_level_rise: True
seismic_hazard: True
slope: True
solar: True
water_risk: True
wsf: True

# Compilation
scan_calculations: True
```

Notes: - `population` = WorldPop. To use GHSL instead, set `population: False` and `ghs_population: True`. - `slope` is derived from `elevation`. If you toggle `slope: True`, also enable `elevation: True` (unless the elevation raster is already on GCS). - `scan_calculations: True` triggers the combined HTML report after all tasks finish. The folder is auto-copied to the city directory.

See `templates/menu.yml` and `tasks/` for the canonical task list.

6 Running a Scan

6.1 Running Tasks

> **Note**
>
> Activate the `cityscan` Python environment before running any `scan` commands.

Tasks are run from the `city-scan` directory or from within `mnt/scan-id` using the general form:

```
scan {TASKNAME} {FLAG}
```

> **Note**
>
> **What is a scan-id?** It is an automatically generated string with the format `yyyy-mm-country-city` (e.g. `2026-02-malta-malta`). It is good practice to always specify the scan-id explicitly when running `scan` commands.

Each task is divided into steps defined by `{FLAG}`:

Flag	What it does
`--collect`	Runs data collection; saves outputs to `02-process-outputs/spatial/` or `/tabular/`
`--analyze`	Runs data analysis; creates output CSVs in `02-process-outputs/tabular/`
`--all`	Runs both `--collect` and `--analyze` for all tasks enabled in `menu.yml`
`--multianalysis`	Runs cross-task analysis (currently flood exposure)

6.1.1 Running All Tasks

To run all tasks enabled in `menu.yml`:

```
scan --all
```

This opens a Terminal UI (TUI) showing per-task progress and logs.

Behavior depends on where you run it from:

- **From `city-scan/` root** — initializes `mnt/scan-id` and copies all necessary folders
- **From inside `mnt/scan-id`** — writes outputs into the local `02-process-outputs/`

To target an existing scan by ID:

```
scan --all --parallel --scan-id 2026-02-malta-malta
```

You will see something like this in your terminal when running in parallel:

```
manila  manila   0   4   32   0
   accessibility      collect  INFO | tasks.accessibility.collection | Downloading all     3
   air_quality        collect  INFO | tasks.air_quality.collection | AOI CRS: EPSG:432     3
   basic_info         collect  city-scan-gee-test                                  37s
  buildings                   waiting (osm)
   burned_area        collect  INFO | tasks.burned_area.collection | Starting burned a     3
  coastal_erosion             waiting (default)
  cyclones                    waiting (default)
  demographics                waiting (default)
  earthquake                  waiting (default)
  elevation                   waiting (gcs)
  wsf                         waiting (gcs)
  oxford                      waiting (default)
  worldpop                    waiting (default)
  fathom                      waiting (gcs)
  flood_events                waiting (default)
  forest                      waiting (gee)
  fwi                         waiting (default)
  gdp_gridded                 waiting (default)
  gdp_sectoral                waiting (default)
  ghs_builtup                 waiting (default)
 [↑↓] select  [Enter] log  [q] quit
 [↑↓] scroll  [Esc] back  [q] quit
```

6.1.2 Running Individual or Multiple Tasks

```
# Single task
scan wsf

# Multiple tasks
scan wsf population forest
```

To run a specific step for one or more tasks:

```
# Collect only
scan wsf --collect

# Analyze only, multiple tasks
scan wsf population green --analyze
```

6.1.3 Other Useful Flags

```
# List all available tasks
scan --list

# Re-run using the city's existing code (no resync from root)
scan --all -k --scan-id 2026-02-malta-malta
```

6.2 Multi-analysis

Some tasks have cross-task analysis scripts — for example, flood-exposure calculations that combine flood and WSF outputs. Run these after all individual tasks have completed:

```
scan --all --multianalysis
```

6.3 Rendering Maps and Charts

After data collection and analysis, generate visualizations using the `--render` flag.

6.3.1 Maps

Applies styling from `source/layers.yml` and writes PNGs to `03-render-output/maps/`:

```
scan --render maps --scan-id 2026-02-malta-malta
```

Maps read spatial data from `02-process-output/spatial/`.

6.3.2 Charts

Writes an HTML file to `tasks/{name}/charts/index.html`:

```
scan --render charts elevation --scan-id 2026-02-malta-malta
```

Charts read CSVs from `02-process-output/tabular/`.

6.3.3 Scan Calculations Reference Sheet

The `scan-calculations` render reads `sections.yml` to determine section order, then assembles all per-task `charts/index.qmd` files into a single combined HTML report:

```
scan --render scan-calculations --scan-id 2026-02-malta-malta
```

The compiled report is saved to `scan-calculations/scan-calculations.html`.

> **Note**
>
> If `scan_calculations: True` is set in `menu.yml`, the scan-calculations folder is automatically copied to the city folder when running `scan --all`.
> See scan-calculations/README.md for details.

Part III

Advanced Workflow

7 Running Multiple City Scans

City Scan supports batch execution across multiple cities using a single configuration file.

7.1 Overview

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Batch mode iterates over a list of city configurations, running each scan sequentially or in parallel depending on available compute resources.

7.2 Batch Configuration

```yaml
# batch_config.yml
scan_type: batch
output_base_dir: "outputs/"
year: 2024

cities:
  - city: "Nairobi"
    country: "KE"
    aoi: "inputs/AOI/nairobi.geojson"

  - city: "Kampala"
    country: "UG"
    aoi: "inputs/AOI/kampala.geojson"

  - city: "Dar es Salaam"
    country: "TZ"
    aoi: "inputs/AOI/dar_es_salaam.geojson"
```

7.3 Running the Batch

```
python run.py --config batch_config.yml --batch
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Scans run sequentially by default. To enable parallel execution:

```
python run.py --config batch_config.yml --batch --workers 3
```

7.4 Progress Tracking

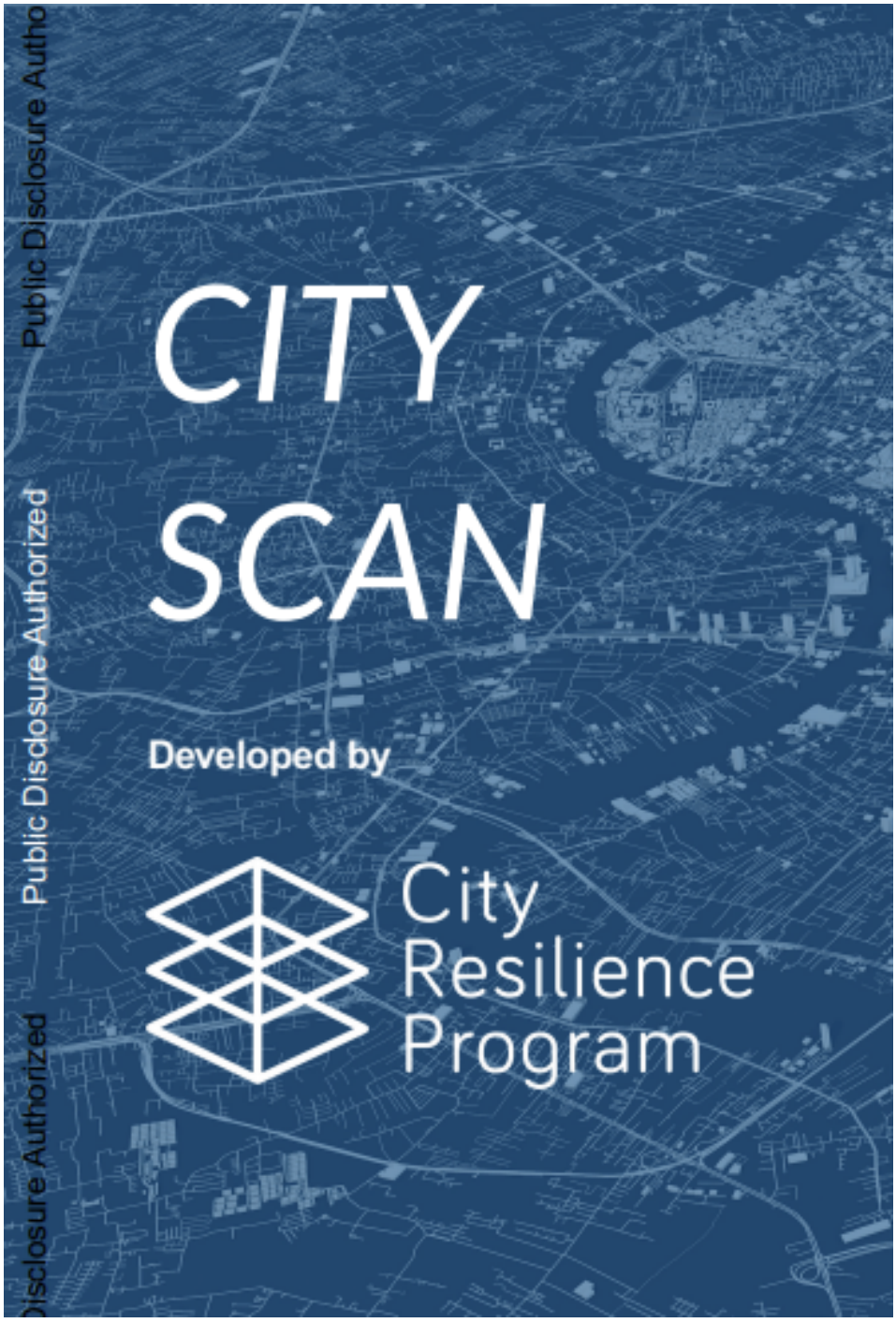


Figure 7.1: Batch scan progress terminal output

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. A progress summary is printed after all scans complete:

```
Batch complete: 3/3 cities succeeded
    Nairobi       (58m 12s)
    Kampala       (1h 03m 47s)
    Dar es Salaam (1h 11m 02s)
```

7.5 Output Structure

```
outputs/
   nairobi/
      maps/
      stats/
      report.html
   kampala/
      ...
   dar_es_salaam/
       ...
```

Table 7.1: Batch run options

Parameter	Default	Description
`--workers`	`1`	Number of parallel scans
`--retry`	`0`	Retry failed scans N times
`--skip-existing`	`false`	Skip cities with existing outputs

> **Warning**
>
> Running too many parallel workers may hit Google Earth Engine API rate limits. Start with `--workers 2` and scale up cautiously.

8 Running a Custom City Scan

A custom scan lets you select specific tasks, override default parameters, and plug in additional data sources beyond the standard suite.

8.1 When to Customize

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Consider a custom scan when:

- You only need a subset of data layers to save time
- You have proprietary or local data to incorporate
- You want non-default resolution or time periods

8.2 Selecting Tasks

```yaml
# custom_config.yml
city: "Jakarta"
country: "ID"
aoi: "inputs/AOI/jakarta.geojson"
output_dir: "outputs/jakarta_custom"
scan_type: custom

tasks:
  - population
  - flood_risk
  - coastal_erosion
```

8.3 Overriding Default Parameters

```yaml
task_overrides:
  population:
    year: 2020
    resolution: 100

  flood_risk:
    return_period: 50   # 50-year flood instead of 100-year
    source: "fathom_v3"
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Overrides are merged with the task defaults, so only specified fields are changed.

8.4 Adding a Custom Data Source

```python
# tasks/custom/my_sensor_data.py
from cityscan.registry import register_task

@register_task
class MySensorDataTask:
    name = "sensor_data"
    inputs = ["aoi"]
    outputs = ["sensor_data.tif"]

    def run(self, config):
        aoi = self.load_aoi(config["aoi"])
        data = self.load_local_raster("data/sensor_grid.tif")
        clipped = self.clip(data, aoi)
        self.write_raster(clipped, "sensor_data.tif")
```

Then reference it in your config:

```yaml
tasks:
  - population
  - sensor_data   # your custom task
```

8.5 Custom Output Directory Layout

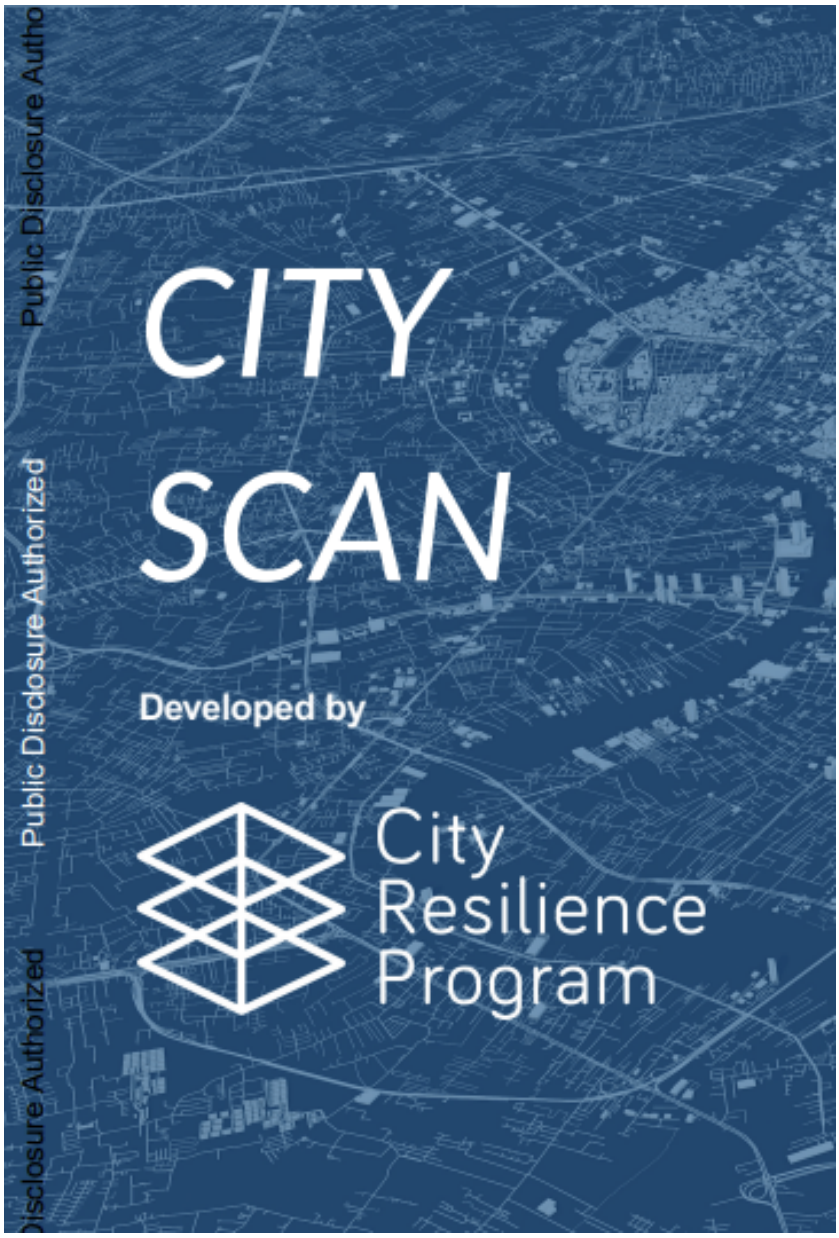


Figure 8.1: Custom scan output structure

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Table 8.1: Custom scan configuration options

Config Key	Type	Description
`tasks`	`list[str]`	Tasks to include
`task_overrides`	`dict`	Per-task parameter overrides
`extra_task_dirs`	`list[str]`	Paths to custom task modules

Tip

Use `--dry-run` to preview which tasks will execute and in what order before committing to a full run.

Part IV

Tasks & Data Layers

9 Tasks & Structure Overview

City Scan is organized around modular **tasks** — each responsible for computing a single data layer. This chapter explains how tasks are structured and how they interact.

9.1 What Is a Task?

Lorem ipsum dolor sit amet, consectetur adipiscing elit. A task is a self-contained Python module that ingests raw data, performs spatial analysis, and writes output files. Each task is independently runnable and cacheable.

```
tasks/
  population.py
  landuse.py
  roads.py
  flood_risk.py
  ...
```

9.2 Task Anatomy

9.2.1 Inputs, Processing, Outputs

```python
# Typical task structure
class PopulationTask:
    name = "population"
    inputs = ["aoi", "worldpop"]
    outputs = ["population_density.tif", "pop_stats.csv"]

    def run(self, config):
        aoi = self.load_aoi(config["aoi"])
        data = self.fetch_worldpop(aoi, config["year"])
        result = self.clip_and_resample(data, aoi)
        self.write_outputs(result, config["output_dir"])
```

9.2.2 Task Metadata

Each task declares metadata used by the orchestrator:

Table 9.1: Task metadata fields

Field	Type	Description
`name`	`str`	Unique task identifier
`inputs`	`list[str]`	Required input data sources
`outputs`	`list[str]`	Files this task produces
`depends_on`	`list[str]`	Tasks that must run first

9.3 Task Categories

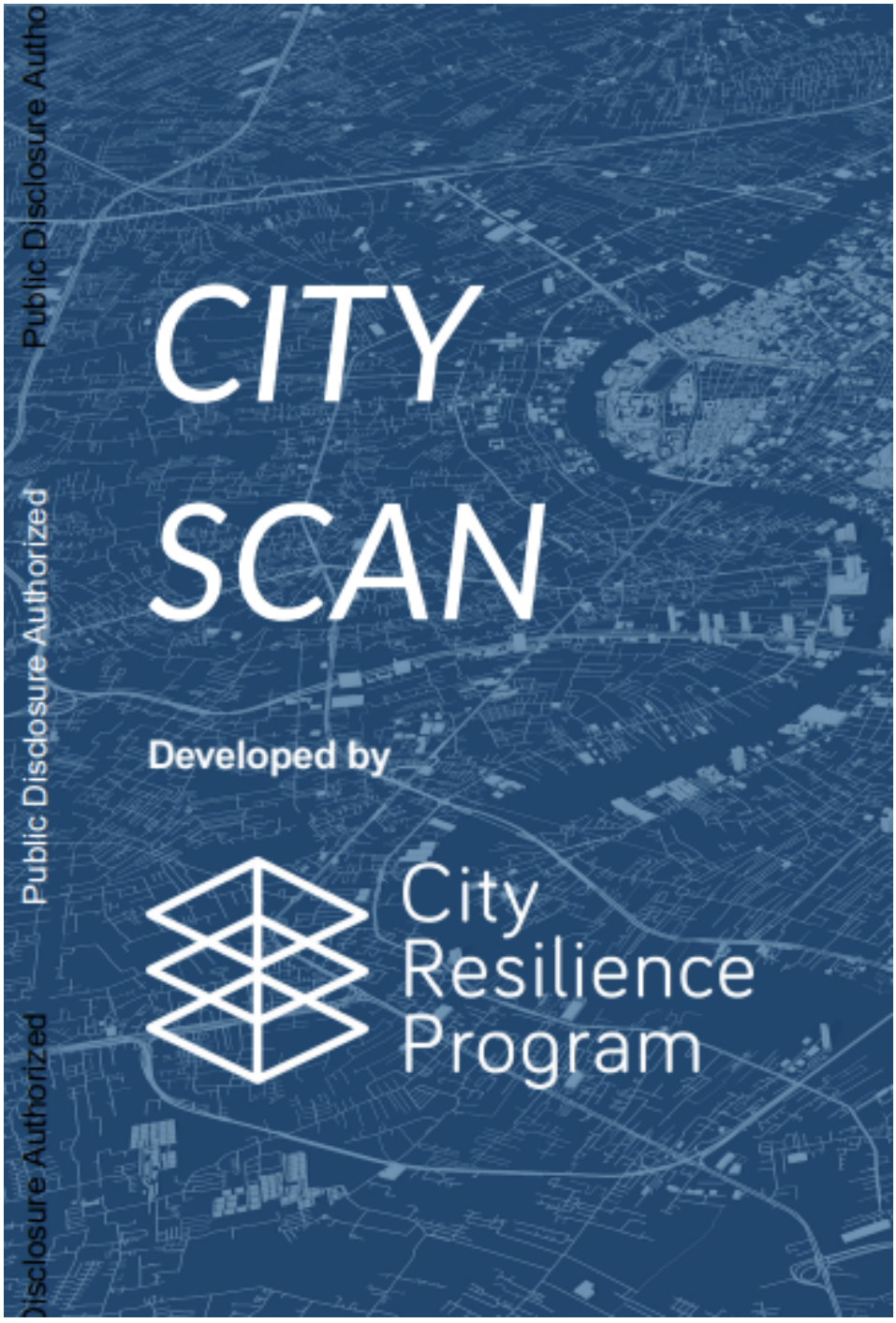


Figure 9.1: Task category overview

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tasks fall into four broad categories:

- **Population** — WorldPop-based density and count estimates
- **Environment** — NDVI, NDWI, land use, and urban heat
- **Infrastructure** — roads, buildings, and accessibility
- **Risk** — flood, coastal erosion, and seismic exposure

9.4 Execution Model

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. The orchestrator resolves task dependencies and runs tasks in topological order, parallelizing independent tasks where possible.

```
# Run all tasks
python run.py --config config.yml

# Run a specific task
python run.py --config config.yml --task population
```

> **Note**
>
> See Data Layer References for a complete list of tasks and their data sources.

10 Data Layer References

Complete reference for all data layers available in City Scan, including source datasets, resolution, and output descriptions.

10.1 Population

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Population layers are derived from WorldPop and LandScan datasets.

Table 10.1: Population data layers

Task ID	Description	Source	Resolution
`population`	Total population count per grid cell	WorldPop	100m
`population_density`	People per km²	WorldPop	100m
`pop_growth`	Annual population growth rate	WorldPop (multi-year)	100m

10.2 Land Use & Environment

```
# Example: computing NDVI
def compute_ndvi(red_band, nir_band):
    return (nir_band - red_band) / (nir_band + red_band)
```

Table 10.2: Land use and environment data layers

Task ID	Description	Source	Resolution
`landuse`	Land use / land cover classification	Dynamic World (GEE)	10m

Task ID	Description	Source	Resolution
`ndvi`	Normalized Difference Vegetation Index	Sentinel-2	10m
`ndwi`	Normalized Difference Water Index	Sentinel-2	10m
`urban_heat`	Land surface temperature	Landsat 8/9	30m

10.3 Infrastructure

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Infrastructure layers use OpenStreetMap as the primary source.

Table 10.3: Infrastructure data layers

Task ID	Description	Source	Resolution
`roads`	Road network density	OpenStreetMap	Vector
`buildings`	Building footprints & density	OpenStreetMap	Vector
`accessibility`	Travel time to services	OpenStreetMap + OTP	100m

10.4 Risk & Hazard

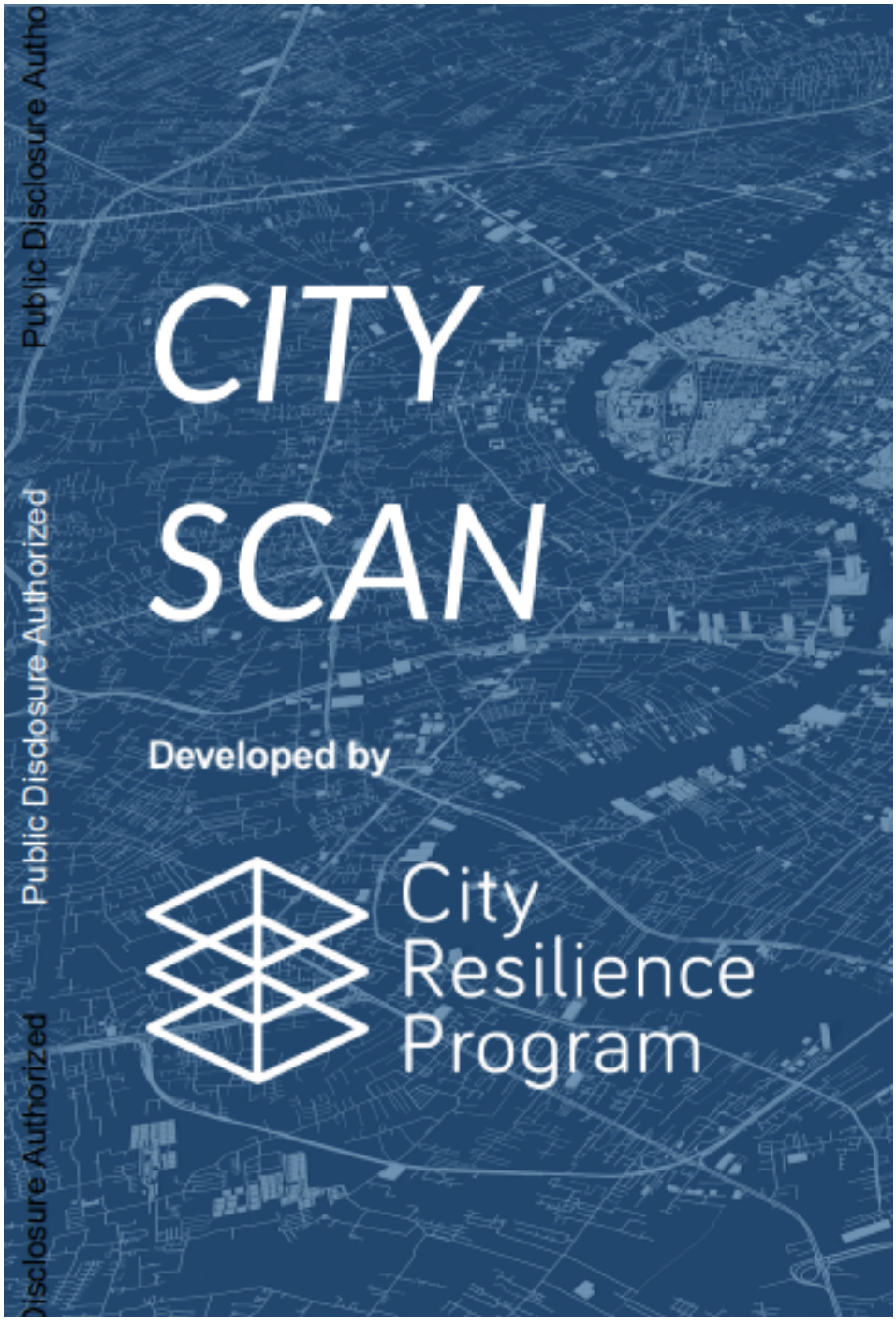


Figure 10.1: Risk layer visualization

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Table 10.4: Risk and hazard data layers

Task ID	Description	Source	Resolution
`flood_risk`	100-year flood extent	Fathom / MERIT Hydro	90m
`coastal_erosion`	Shoreline change rate	CoastalDEM + Sentinel-1	30m
`seismic`	Peak ground acceleration	GSHAP	~10km

10.5 Adding a Custom Data Layer

```
# Register a new task
from cityscan.registry import register_task

@register_task
class MyCustomTask:
    name = "my_custom_layer"
    inputs = ["aoi"]
    outputs = ["my_custom_layer.tif"]

    def run(self, config):
        # implementation here
        pass
```

> **Tip**
>
> Custom tasks can be placed in the `tasks/custom/` directory and will be auto-discovered at runtime.

Part V

Understanding the Repo

11 Repository Structure

An orientation to the City Scan codebase — what lives where and why.

11.1 Top-level Layout

```
city-scan/
  cityscan/          # core Python package
  tasks/             # task modules
  inputs/            # user-supplied inputs (AOI, local data)
  outputs/           # generated scan outputs (git-ignored)
  scripts/           # utility scripts
  tests/             # test suite
  docs/              # this documentation
  pyproject.toml     # package metadata & dependencies
  run.py             # CLI entry point
  orchestrator.sh    # shell-based orchestrator (legacy)
```

11.2 Core Package: `cityscan/`

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

```
cityscan/
  __init__.py
  config.py          # config loading and validation
  registry.py        # task registration and discovery
  runner.py          # task execution and dependency resolution
  gee.py             # Google Earth Engine helpers
  io.py              # file I/O utilities
  report.py          # HTML report generation
```

11.2.1 Key Modules

Table 11.1: Core module responsibilities

Module	Responsibility
`config.py`	Load and validate `config.yml`
`registry.py`	Discover and register task classes
`runner.py`	Resolve dependencies, schedule, and run tasks
`gee.py`	Authenticate and interact with GEE

11.3 Tasks Directory: `tasks/`

```
tasks/
   population.py
   landuse.py
   roads.py
   flood_risk.py
   coastal_erosion.py
   custom/            # user-defined custom tasks
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Each file in `tasks/` is a self-contained module that is auto-discovered by the registry at startup.

11.4 Inputs & Outputs

```
inputs/
   AOI/               # place your GeoJSON boundary files here

outputs/               # created automatically; git-ignored
   <city_name>/
       maps/
       stats/
       report.html
```

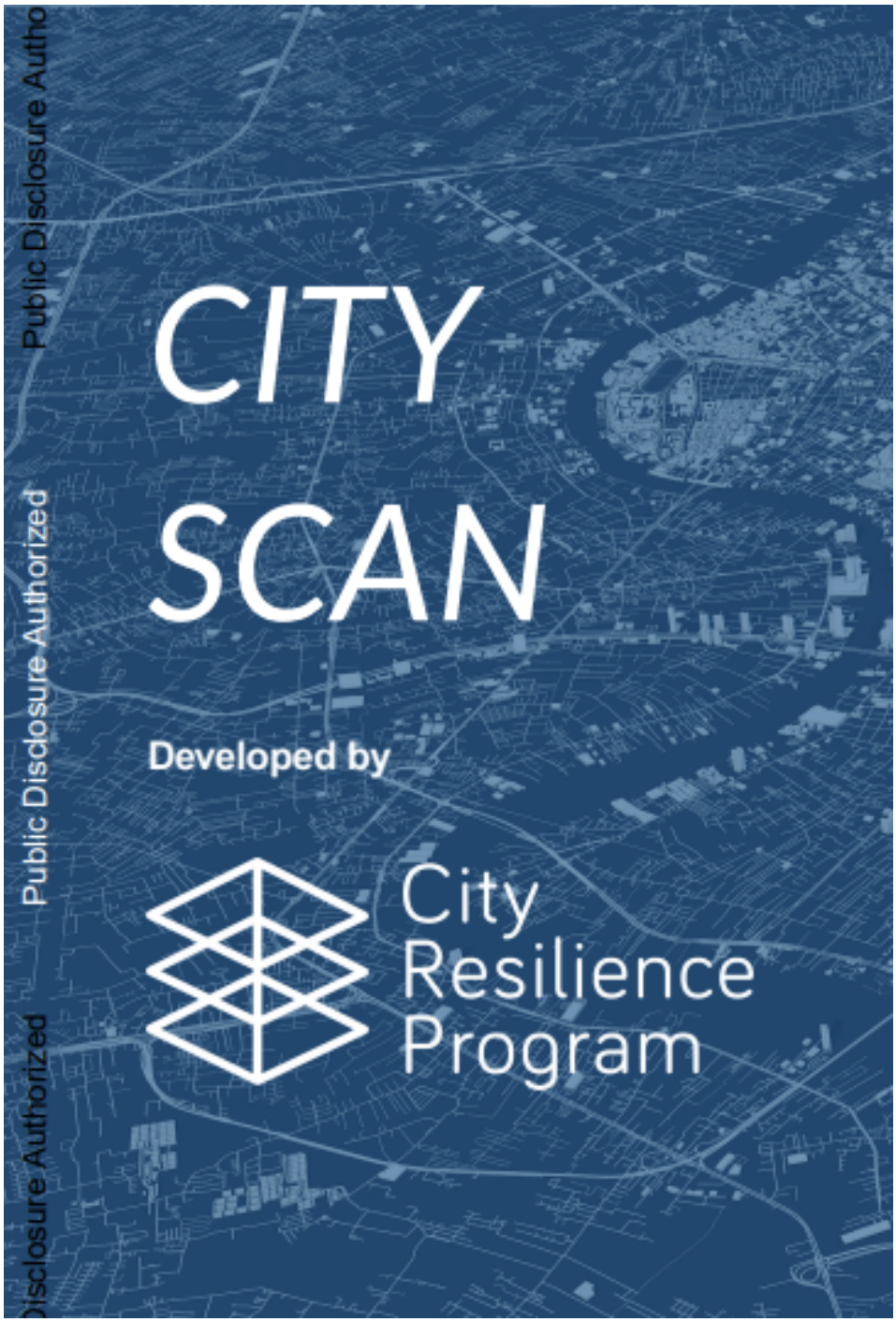


Figure 11.1: Repository structure diagram

11.5 Tests

```
pytest tests/            # run the full test suite
pytest tests/unit/       # unit tests only
pytest tests/integration/ # integration tests (requires GEE)
```

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

> **Note**
>
> For how tasks connect to each other, see Task Dependency.

12 Task Dependency

City Scan tasks form a directed acyclic graph (DAG). Understanding dependencies helps you predict execution order and debug failures.

12.1 Dependency Graph

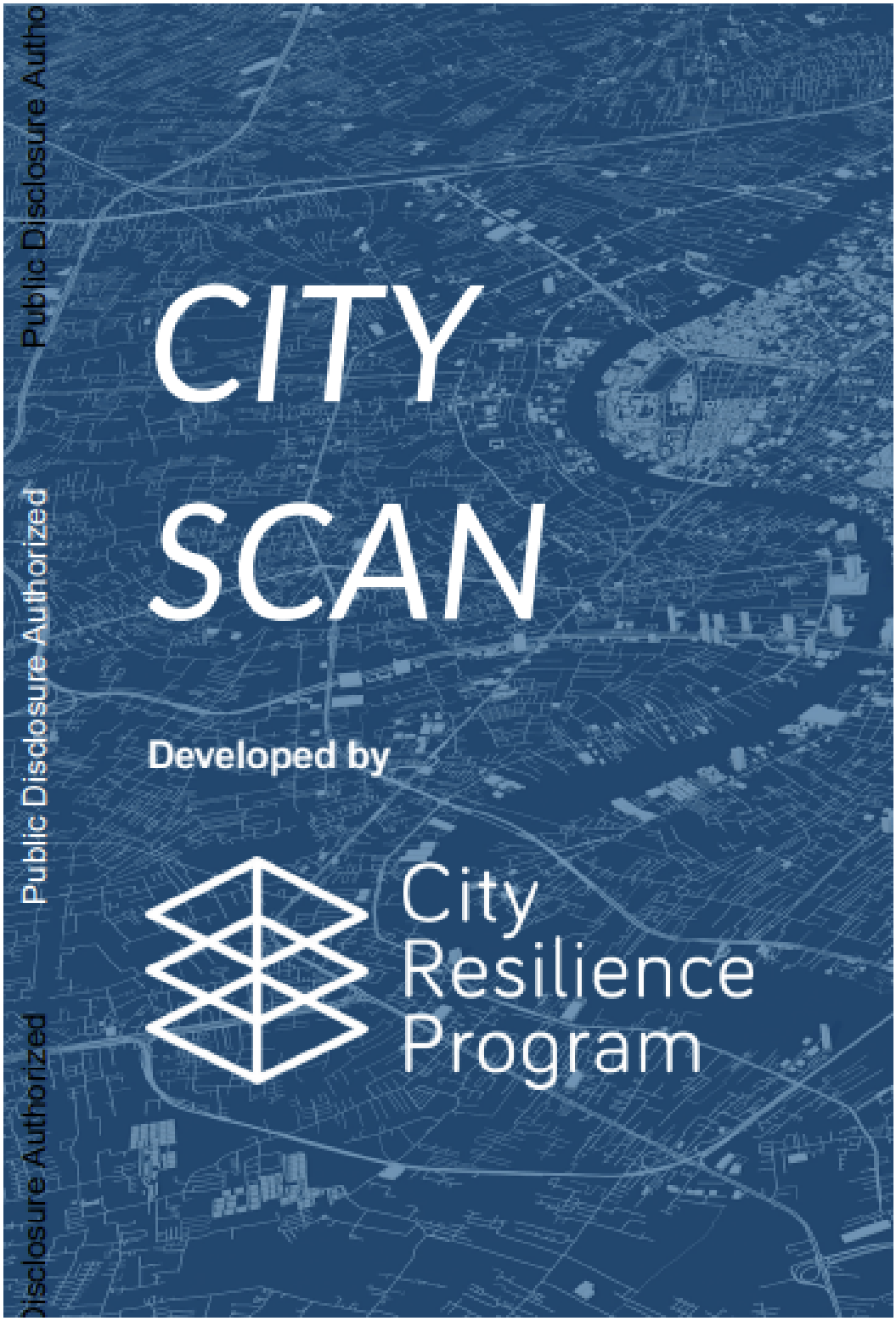


Figure 12.1: Task dependency DAG

Lorem ipsum dolor sit amet, consectetur adipiscing elit. The orchestrator builds this graph at startup by reading each task's `depends_on` declaration.

12.2 How Dependencies Are Declared

```
class AccessibilityTask:
    name = "accessibility"
    depends_on = ["roads", "population"]  # must run after these

    def run(self, config):
        roads = self.load_output("roads")
        pop   = self.load_output("population")
        # ... compute accessibility
```

12.3 Execution Order Example

For the default task suite, the orchestrator produces the following topological order:

```
1. aoi_prep
2. population          (depends on: aoi_prep)
3. landuse             (depends on: aoi_prep)
4. roads               (depends on: aoi_prep)
5. ndvi                (depends on: aoi_prep)
6. flood_risk          (depends on: aoi_prep)
7. accessibility       (depends on: roads, population)
8. urban_heat          (depends on: landuse, ndvi)
9. report              (depends on: all above)
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tasks at the same level can be parallelized.

12.4 Dependency Table

Table 12.1: Task dependency matrix

Task	Depends On
`population`	`aoi_prep`
`roads`	`aoi_prep`
`landuse`	`aoi_prep`
`flood_risk`	`aoi_prep`
`accessibility`	`roads`, `population`
`urban_heat`	`landuse`, `ndvi`
`report`	all tasks

12.5 Handling Failures

```
# Resume a scan after a failed task
python run.py --config config.yml --resume
```

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. With `--resume`, the runner skips tasks whose output files already exist and only re-runs tasks downstream of the failure point.

12.6 Visualizing the DAG

```
from cityscan.runner import Runner

runner = Runner.from_config("config.yml")
runner.visualize_dag("dag.png")   # saves a PNG of the dependency graph
```

> **Tip**
>
> Use `python run.py --dry-run --config config.yml` to print the resolved execution order without running any tasks.

Part VI

Case Studies & Galleries

13 Case Study: Standard Scan — Rabat, Morocco

This case study walks through a complete standard City Scan for Rabat, Morocco, demonstrating the end-to-end workflow and key findings.

13.1 City Context

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Rabat is the capital of Morocco, with a metropolitan population of approximately 1.9 million. The city faces urban growth pressures, coastal flood risk, and heat island effects.

Table 13.1: Rabat scan metadata

Attribute	Value
Country	Morocco
Population	~1.9 million
Area (AOI)	~117 km²
Scan date	March 2024
Tasks run	14

13.2 AOI Definition

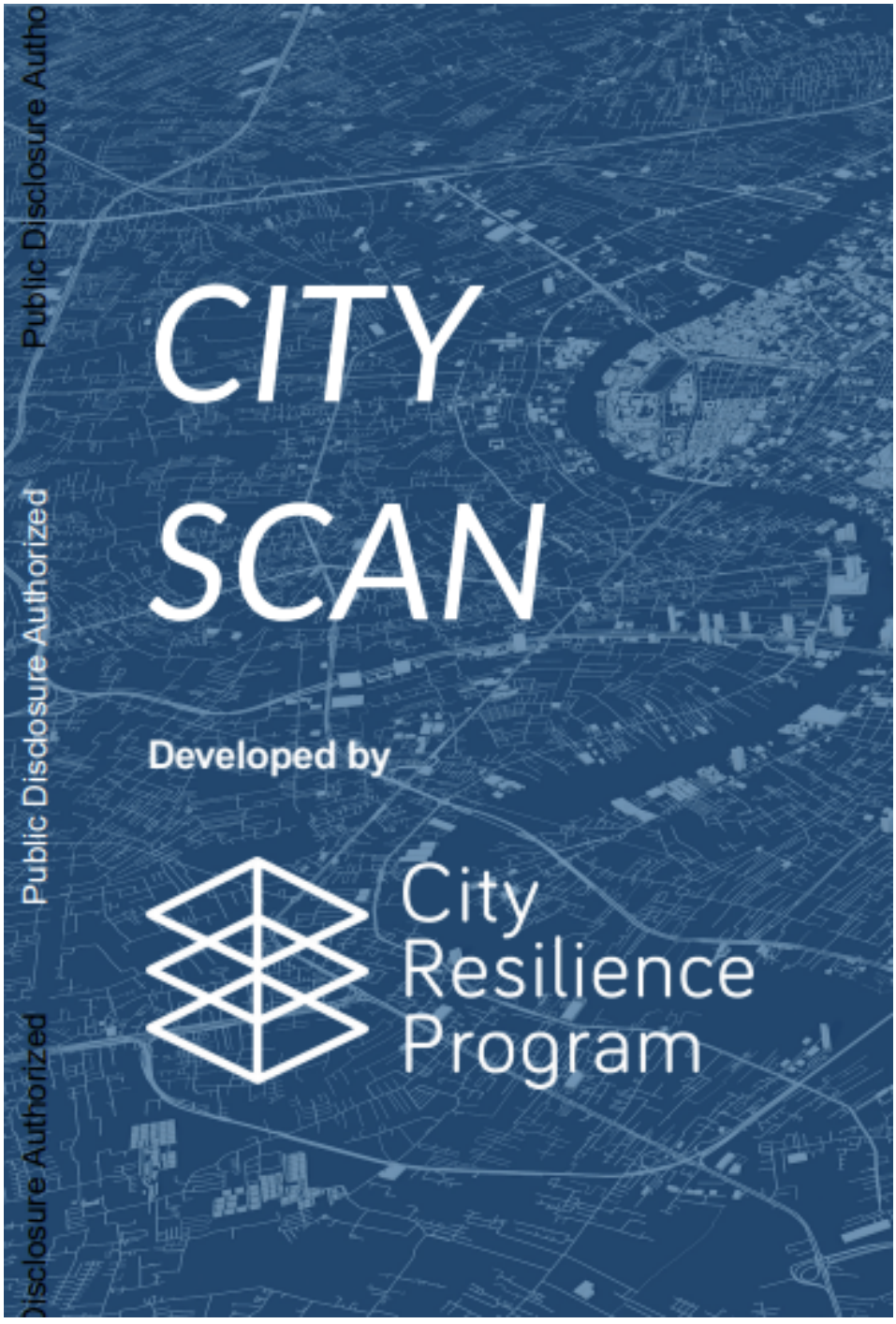


Figure 13.1: Rabat area of interest

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. The AOI was defined using the official administrative boundary of the Rabat-Salé-Kénitra region, clipped to the primary urban core.

```
# AOI file used
inputs/AOI/rabat_urban_core.geojson
```

13.3 Running the Scan

```
python run.py --config configs/rabat.yml
```

```
# configs/rabat.yml
city: "Rabat"
country: "MA"
aoi: "inputs/AOI/rabat_urban_core.geojson"
output_dir: "outputs/rabat"
year: 2024
scan_type: standard
```

13.4 Key Findings

13.4.1 Population Distribution

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Population is concentrated in the Agdal and Hassan neighborhoods, with density exceeding 15,000 people/km^2.

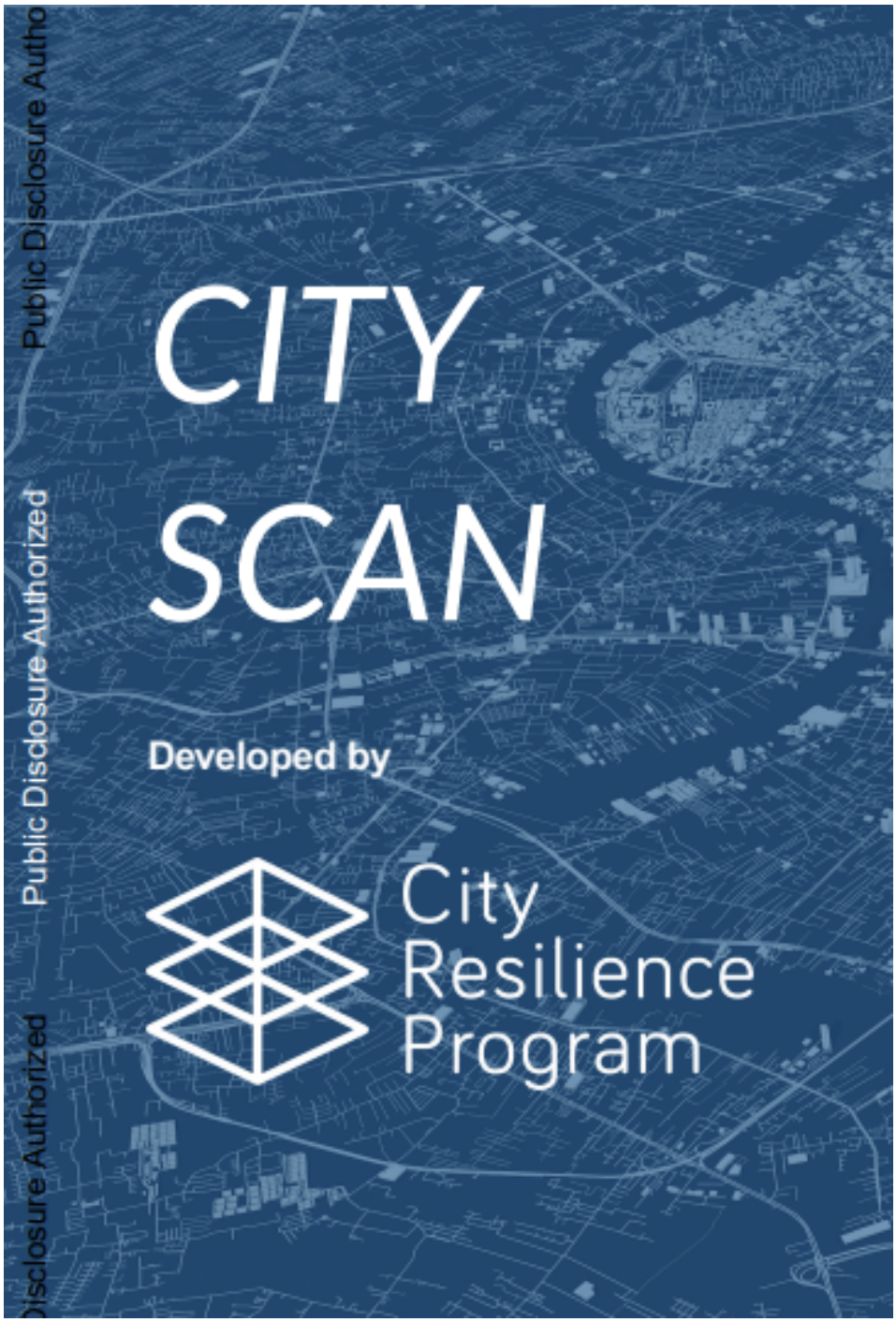


Figure 13.2: Rabat population density map

13.4.2 Flood Risk

Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Approximately 8% of the built environment falls within the 100-year flood extent, concentrated along the Bouregreg River corridor.

13.4.3 Land Use

```
# Summary statistics from outputs/rabat/stats/landuse_stats.csv
landuse_summary = {
    "Urban/Built-up": 62.3,
    "Vegetation": 18.7,
    "Bare soil": 11.4,
    "Water": 7.6,
}
```

13.5 Outputs

All scan outputs are available at `outputs/rabat/`. The interactive report can be viewed at `outputs/rabat/report.html`.

> **Note**
>
> For a multi-city batch scan case study, see Case Study: Multiple Scans — Philippines.

14 Case Study: Multiple Scans — Philippines

This case study demonstrates running City Scan across multiple Philippine cities simultaneously using batch mode.

14.1 Project Context

Lorem ipsum dolor sit amet, consectetur adipiscing elit. The World Bank Philippines office requested City Scans for five secondary cities to support climate resilience planning. Batch mode was used to process all cities in a single run.

14.2 Cities Covered

Table 14.1: Philippine cities included in the batch scan

City	Region	Population	AOI Area
Cebu	Central Visayas	964,169	315 km²
Davao	Davao Region	1,776,949	2,444 km²
Iloilo	Western Visayas	457,626	78 km²
Cagayan de Oro	Northern Mindanao	728,402	488 km²
Zamboanga	Zamboanga Peninsula	977,234	1,483 km²

14.3 Batch Configuration

```yaml
# configs/philippines_batch.yml
scan_type: batch
output_base_dir: "outputs/philippines"
year: 2023

cities:
  - city: "Cebu"
```

```
    country: "PH"
    aoi: "inputs/AOI/cebu.geojson"
  - city: "Davao"
    country: "PH"
    aoi: "inputs/AOI/davao.geojson"
  - city: "Iloilo"
    country: "PH"
    aoi: "inputs/AOI/iloilo.geojson"
  - city: "Cagayan de Oro"
    country: "PH"
    aoi: "inputs/AOI/cdo.geojson"
  - city: "Zamboanga"
    country: "PH"
    aoi: "inputs/AOI/zamboanga.geojson"
```

14.4 Running the Batch

```
python run.py --config configs/philippines_batch.yml --batch --workers 2
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Two parallel workers were used to balance GEE API rate limits with execution speed. Total batch runtime was approximately 6 hours.

14.5 Key Findings Across Cities

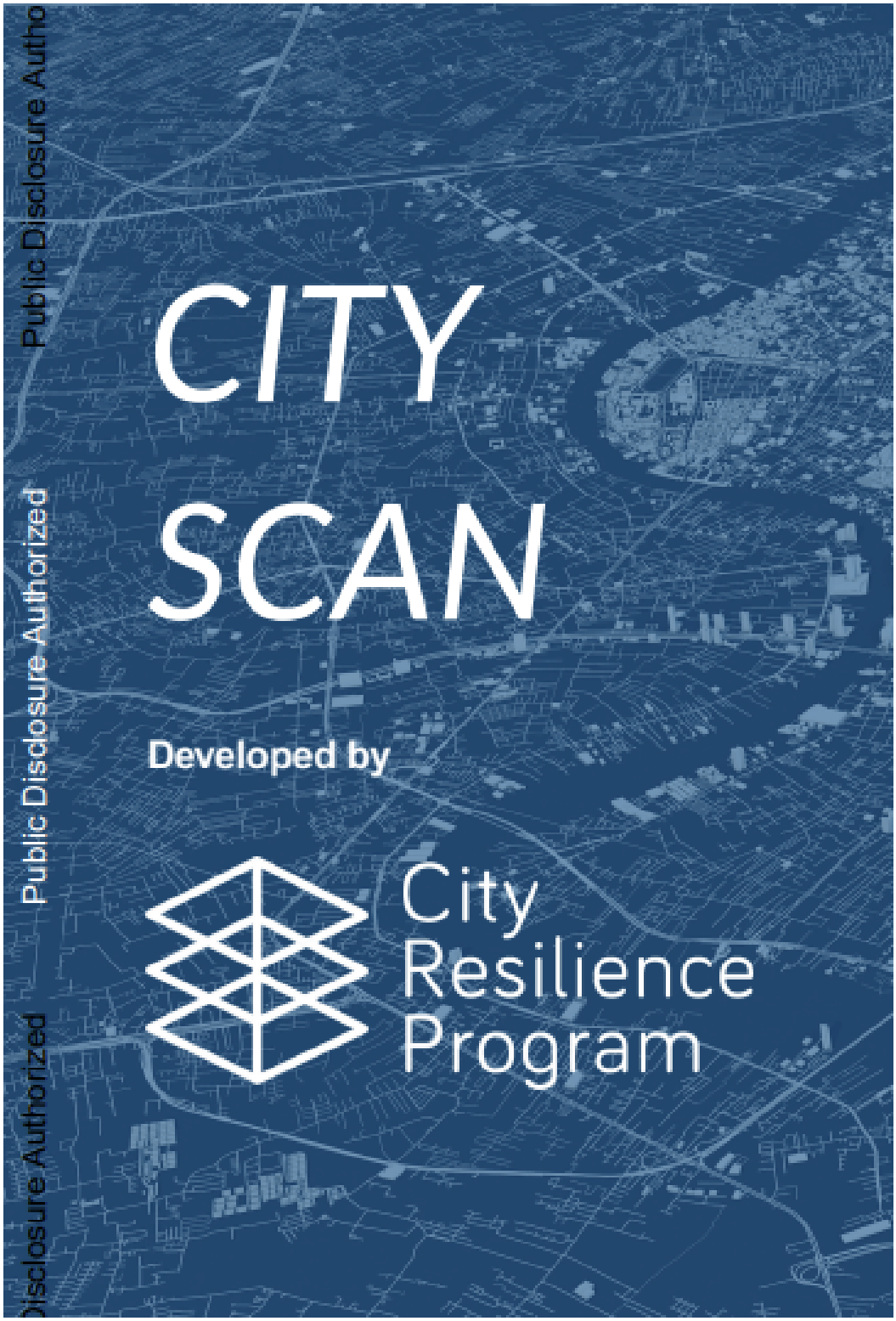


Figure 14.1: Flood risk comparison across Philippine cities

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

14.5.1 Coastal Flood Risk

All five cities have significant coastal exposure. Zamboanga showed the highest percentage of built-up area within the 100-year flood extent at 23%.

14.5.2 Urban Heat Islands

```
# Comparative heat island intensity (°C above rural baseline)
heat_island = {
    "Cebu": 3.2,
    "Davao": 2.8,
    "Iloilo": 2.1,
    "Cagayan de Oro": 2.6,
    "Zamboanga": 3.5,
}
```

14.6 Lessons Learned

Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Key takeaways from this multi-city deployment:

- Pre-validate all AOI files before starting the batch to avoid mid-run failures.
- Use `--workers 2` as the safe default for GEE-heavy task suites.
- Monitor GEE quota usage via the GEE Console during long runs.

> **Note**
>
> Configuration files for all five cities are available in `configs/examples/philippines/`.

Part VII

Collaborating

15 Git Guide

A practical Git reference for City Scan contributors, covering branching strategy, commit conventions, and common workflows.

15.1 Branching Strategy

Lorem ipsum dolor sit amet, consectetur adipiscing elit. City Scan uses a trunk-based development model with short-lived feature branches.

```
main                    # stable, always deployable
  feature/<name>      # new features or improvements
  fix/<name>          # bug fixes
  docs/<name>         # documentation-only changes
```

15.2 Cloning the Repository

```
git clone https://github.com/cityresilience/city-scan.git
cd city-scan
git checkout -b feature/my-new-feature
```

15.3 Commit Message Conventions

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Follow the Conventional Commits format:

```
<type>(<scope>): <short description>

[optional body]

[optional footer]
```

Table 15.1: Conventional commit types

Type	When to Use
`feat`	New feature or data layer
`fix`	Bug fix
`docs`	Documentation changes only
`refactor`	Code restructuring, no behavior change
`test`	Adding or updating tests
`chore`	Maintenance (deps, CI, config)

15.3.1 Examples

```
git commit -m "feat(tasks): add seismic hazard layer"
git commit -m "fix(gee): handle expired credentials gracefully"
git commit -m "docs(setup): clarify GEE authentication steps"
```

15.4 Pull Request Workflow

```
# 1. Create a branch
git checkout -b fix/flood-risk-clipping

# 2. Make changes and commit
git add tasks/flood_risk.py
git commit -m "fix(tasks): correct AOI clipping in flood risk task"

# 3. Push and open a PR
git push -u origin fix/flood-risk-clipping
gh pr create --fill
```

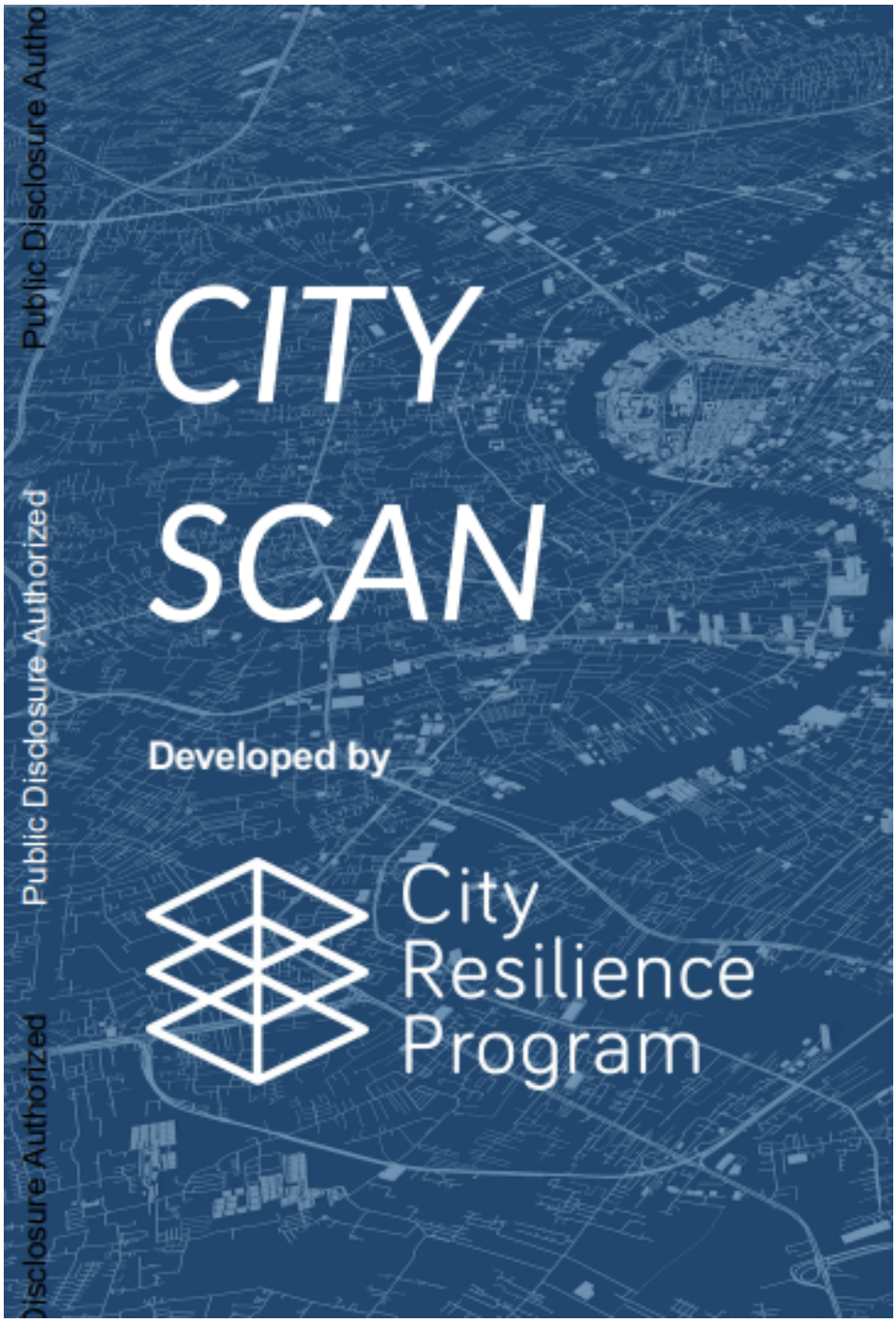


Figure 15.1: Pull request checklist on GitHub

15.5 Keeping Your Branch Up to Date

```
git fetch origin
git rebase origin/main
```

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

> **Warning**
>
> Avoid `git merge main` into feature branches — use rebase to keep a clean linear history.

> **Note**
>
> For opening issues, see Reporting Issues. For contributing code, see Contributing.

16 Reporting Issues

How to report bugs, request features, and ask questions about City Scan.

16.1 Where to Report

Lorem ipsum dolor sit amet, consectetur adipiscing elit. All issues are tracked on the GitHub repository:

> **GitHub Issues:** `https://github.com/cityresilience/city-scan/issues`

16.2 Before You Open an Issue

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Before filing a new issue, please:

1. **Search existing issues** — your problem may already be reported or fixed.
2. **Check the documentation** — especially Setting Up for environment problems.
3. **Reproduce the issue** — confirm it happens on a clean install.

16.3 Bug Reports

Use the **Bug Report** template when something is broken. Include:

```
## Description
A clear description of what went wrong.

## Steps to Reproduce
1. Run `python run.py --config config.yml`
2. Observe error in `outputs/city/scan.log`

## Expected Behavior
The scan should complete successfully.
```

```
## Actual Behavior
Task `flood_risk` fails with `KeyError: 'return_period'`.

## Environment
- OS: macOS 14.4
- Python: 3.11.6
- City Scan version: 0.9.2
```

16.4 Feature Requests

Use the **Feature Request** template to propose new data layers, CLI options, or workflow improvements.

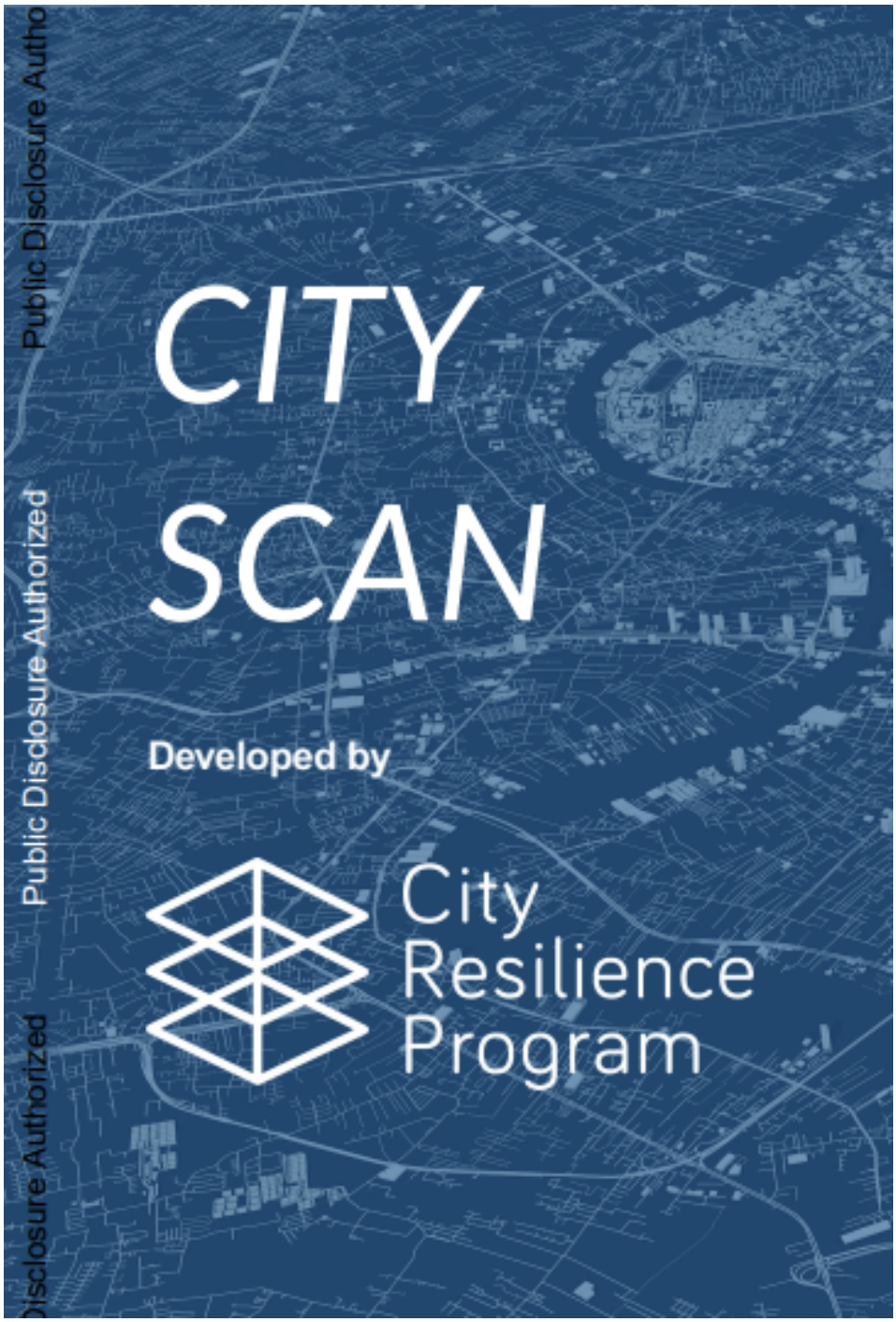


Figure 16.1: GitHub issue tracker showing a mix of bug reports and feature requests

16.5 Sharing Log Files

```
# Find your scan log
cat outputs/<city>/scan.log

# Or attach the whole log to the issue
```

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Redact any sensitive paths or credentials before sharing logs.

16.6 Issue Labels

Table 16.1: GitHub issue labels

Label	Meaning
`bug`	Confirmed defect
`enhancement`	New feature or improvement
`documentation`	Docs gap or correction
`good first issue`	Suitable for new contributors
`help wanted`	Maintainers welcome external PRs

> **Tip**
>
> For quick questions, open a **Discussion** instead of an issue — discussions are better for open-ended conversations.

17 Contributing

We welcome contributions to City Scan — from new data layers to documentation improvements.

17.1 Types of Contributions

Lorem ipsum dolor sit amet, consectetur adipiscing elit:

- **New tasks** — add a data layer not currently in the suite
- **Bug fixes** — resolve issues in existing tasks or infrastructure
- **Documentation** — improve this cookbook
- **Tests** — expand test coverage
- **Performance** — optimize slow tasks or reduce GEE API calls

17.2 Development Setup

```
git clone https://github.com/cityresilience/city-scan.git
cd city-scan
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
pre-commit install
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. The `[dev]` extras install linting, formatting, and test tools.

17.3 Code Style

City Scan uses Ruff for linting and formatting.

```
ruff check .          # lint
ruff format .         # format
```

Pre-commit hooks enforce these checks on every commit. To run manually:

```
pre-commit run --all-files
```

17.4 Writing a New Task

```
# tasks/my_new_layer.py
from cityscan.registry import register_task
from cityscan.base import BaseTask

@register_task
class MyNewLayerTask(BaseTask):
    name = "my_new_layer"
    description = "Computes lorem ipsum data layer."
    inputs = ["aoi"]
    outputs = ["my_new_layer.tif"]
    depends_on = []

    def run(self, config: dict) -> None:
        aoi = self.load_aoi(config["aoi"])
        result = self._compute(aoi)
        self.write_raster(result, "my_new_layer.tif")

    def _compute(self, aoi):
        # implementation here
        pass
```

Add a corresponding test in `tests/unit/test_my_new_layer.py`.

17.5 Pull Request Checklist

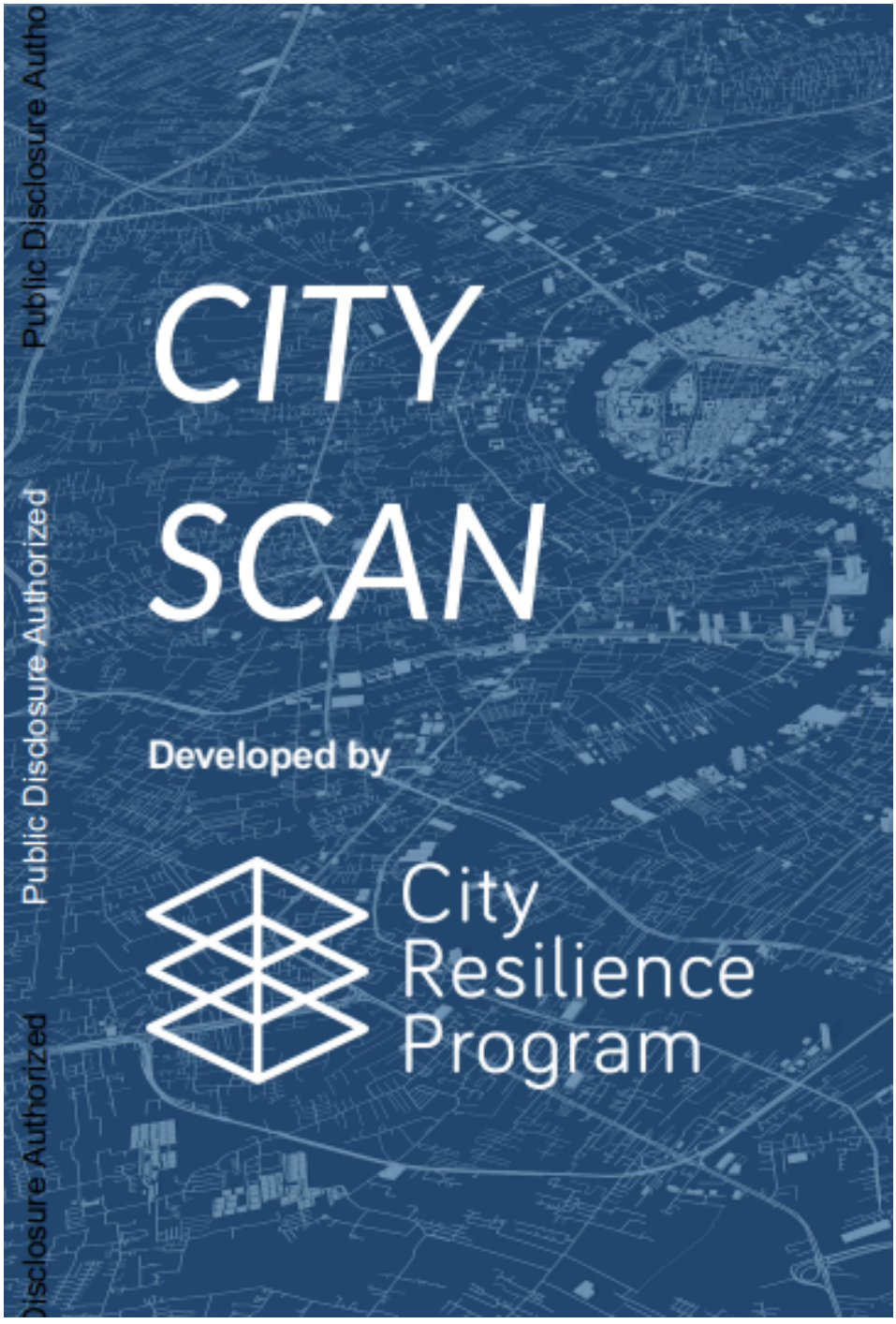


Figure 17.1: Pull request checklist

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

- ☐ Tests pass: `pytest tests/`
- ☐ Linting passes: `ruff check .`
- ☐ New task is registered and documented in Data Layer References
- ☐ PR title follows Conventional Commits format
- ☐ No secrets or credentials committed

17.6 Review Process

Table 17.1: Pull request review stages

Stage	Who	Typical Duration
Automated checks (CI)	GitHub Actions	$<$ 5 min
Code review	Core maintainer	1–3 days
Merge	Core maintainer	After approval

> **Note**
>
> See Git Guide for branching and commit conventions.

18 Sharing the Guide

How to share, publish, and distribute the City Scan Cookbook with your team and stakeholders.

18.1 Rendered Formats

Lorem ipsum dolor sit amet, consectetur adipiscing elit. The cookbook is built with Quarto and can be rendered to HTML, PDF, or both.

```
# Render HTML (default)
quarto render

# Render PDF
quarto render --to pdf

# Render both
quarto render --to html,pdf
```

Output is written to `_book/`.

18.2 Publishing to GitHub Pages

```
quarto publish gh-pages
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. This command builds the book and pushes the rendered HTML to the `gh-pages` branch. The site is then accessible at:

`https://cityresilience.github.io/city-scan/`

18.3 Publishing to Quarto Pub

```
quarto publish quarto-pub
```

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. You will be prompted to sign in to your Quarto Pub account on first use.

18.4 Sharing the PDF

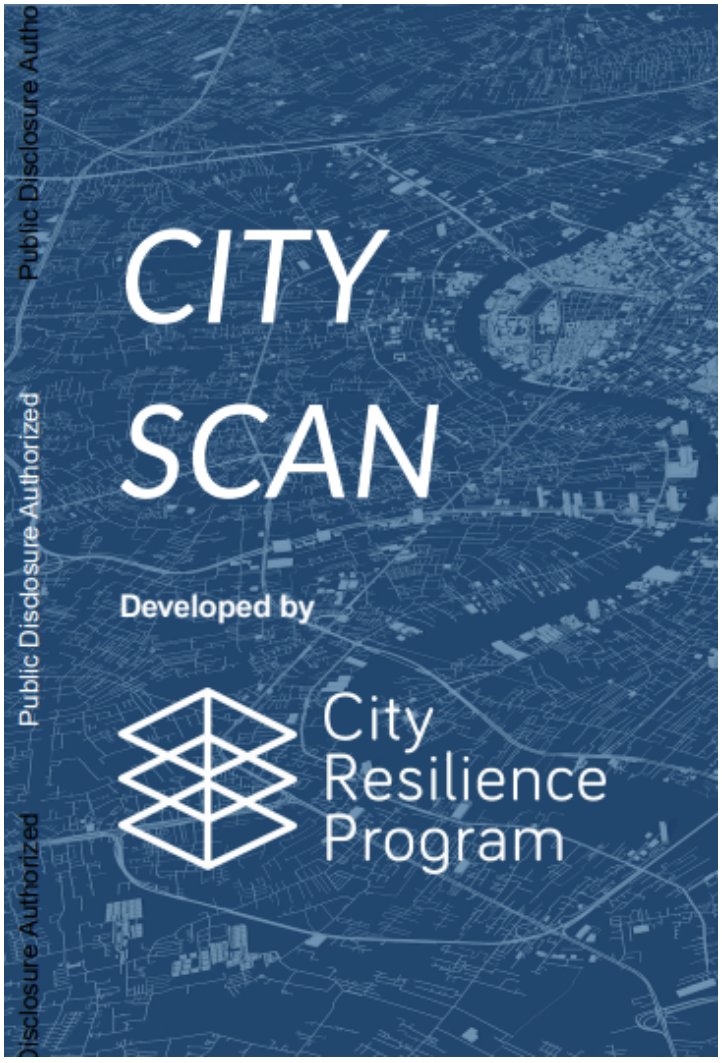


Figure 18.1: PDF export of the City Scan Cookbook

After rendering to PDF:

```
open _book/City-Scan-Cook-Book.pdf    # macOS
```

Table 18.1: Guide distribution options

Distribution Method	Audience	Notes
GitHub Pages (HTML)	Technical users	Free, auto-updates on push
Quarto Pub (HTML)	General stakeholders	Free, public URL
PDF via email	Partners / clients	Snapshot in time
PDF on SharePoint	Internal World Bank	Behind org auth

18.5 Embedding in Other Sites

```
<!-- Embed a specific chapter as an iframe -->
<iframe
  src="https://cityresilience.github.io/city-scan/setting-up.html"
  width="100%"
  height="600px"
  frameborder="0">
</iframe>
```

18.6 Keeping the Guide Up to Date

```
# Set up GitHub Actions to auto-publish on every push to main
# .github/workflows/publish.yml
```

```
on:
  push:
    branches: [main]

jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - uses: quarto-dev/quarto-actions/publish@v2
        with:
          target: gh-pages
```

> **Tip**
>
> For feedback on the guide itself, open a `documentation` issue on GitHub — see Reporting Issues.

19 Editing This Reference

A step-by-step guide for team members who want to update or extend this technical reference. Covers the editing workflow, branch conventions, and how to publish changes to GitHub Pages.

19.1 Prerequisites

- **Quarto** installed (quarto.org)
- **Git** installed and authenticated with GitHub
- Write access to the `cityresilience/city-scan` repository

19.2 Repository Structure

Documentation lives in the `docs/` folder of the main `city-scan` repository. Each page is a Quarto Markdown file (`.qmd`):

```
city-scan/
  docs/
      _quarto.yml         # book structure and settings
      index.qmd           # landing page
      *.qmd               # one file per chapter
      references.bib      # bibliography
      image/              # shared images
```

The compiled website is published to the `gh-pages` branch and served at `https://cityresilience.github.io/c` `scan/`.

19.3 Editing Workflow

19.3.1 1. Switch to `new_docs`

All documentation edits happen on `new_docs` — never commit directly to `main` or `gh-pages`.

```
git checkout new_docs
git pull origin new_docs      # always pull latest before editing
```

19.3.2 2. Edit `.qmd` files

Open the relevant `.qmd` file in `docs/` and edit it. Quarto Markdown is standard Markdown with optional callouts, cross-references, code blocks, and diagrams.

To preview changes live in your browser:

```
cd docs
quarto preview
```

This starts a local server (usually at `http://localhost:4848`) that hot-reloads as you save.

19.3.3 3. Add a new page (optional)

1. Create a new `.qmd` file in `docs/`, e.g. `docs/my-new-page.qmd`
2. Start the file with an H1 heading: `# My New Page`
3. Register it in `docs/_quarto.yml` under the appropriate `part:` section:

```
- part: "Collaborating"
  chapters:
    - editing-this-reference.qmd
    - my-new-page.qmd
```

19.3.4 4. Commit and push to `new_docs`

```
# Stage only docs source files - never stage docs/_book/
git add docs/my-new-page.qmd docs/_quarto.yml
git commit -m "docs: add page on <topic>"
git push origin new_docs
```

> **Warning**
>
> Do **not** stage or commit anything inside `docs/_book/`. That folder contains compiled site output and is excluded by `docs/.gitignore`. Committing it to `new_docs` will bloat the repository with binary files.

19.4 Publishing Workflow

Once edits are committed to `new_docs`, publish to GitHub Pages from the `docs/` directory:

```
cd docs
quarto publish gh-pages
```

Quarto will:

1. Re-render only changed pages (`freeze: true` skips unchanged code outputs)
2. Commit the compiled site to the local `gh-pages` branch
3. Push `gh-pages` to `origin` — GitHub Pages auto-deploys within ~1 minute

When prompted:

```
Update site at https://cityresilience.github.io/city-scan/? (Y/n)
```

Type `Y` and press Enter.

> **Note**
>
> **First-time setup on a new machine:** If you get an `HTTP 400` error during the push step, run this once from the repo root, then re-publish:
>
> ```
> git config http.postBuffer 524288000
> ```
>
> This increases the git push buffer to 500 MB, which is needed for the initial upload of all site assets.

19.5 Branch Overview

Branch	Purpose	Who touches it
`main`	Stable code and configuration	Maintainers via PR
`new_docs`	Documentation source (`.qmd` files)	Editors directly
`gh-pages`	Compiled website — managed by Quarto	**Never edit manually**

19.6 Quick Reference

```
# Pull latest docs before editing
git checkout new_docs && git pull origin new_docs

# Start local preview
cd docs && quarto preview

# Commit documentation edits (from repo root)
git add docs/<changed-files>
git commit -m "docs: <short description>"
git push origin new_docs

# Publish to GitHub Pages (from docs/)
cd docs && quarto publish gh-pages
```

> **Note**
>
> For reporting content issues or suggesting structural changes, see Reporting Issues. For contributing to the codebase itself, see Contributing.

References